

中国研究生操作系统开源创新大赛

项目说明书

作品名称:	基于深度强化学习的云一 边一端异构计算资源管理 调度方法
参赛队伍:	操作系统创新小分队
完成日期:	2026年10月

目录

第 1 章 绪 论.....	1
1.1 目的与意义.....	1
1.2 项目背景.....	1
第 2 章 技术 理 论.....	3
2.1 设计思想.....	3
2.1.1 任务与资源的数学建模.....	3
2.1.2 以统一物理规则保障公平比较.....	3
2.1.3 学习与约束的职责划分.....	4
2.2 技术路线.....	4
2.2.1 环境状态、就绪集合与动作屏蔽.....	5
2.2.2 HEFT 残差策略与 MaskablePPO 训练.....	5
2.2.3 相对 HEFT 的终局奖励.....	6
2.2.4 多样初解与合法搜索精修.....	6
2.2.5 Adaptive 候选组合、容量安全与软预算.....	7
2.2.6 统一接口与实验复现.....	8
2.3 代码原创说明.....	8
2.3.1 第三方库及其引用方式.....	8
2.3.2 算法依据与项目实现的界限.....	9
第 3 章 项目介绍.....	11
3.1 功能介绍.....	11
3.1.1 任务与资源配置.....	11
3.1.2 统一调度接口与合法输出.....	11
3.1.3 可训练策略与质量控制.....	12
3.1.4 评测与诊断.....	12
3.1.5 运行、复现与工程交付.....	13
3.2 软件界面.....	13
3.2.1 命令行调度操作.....	13
3.2.2 机器可读结果与资源甘特图.....	14
3.2.3 本地演示附件的范围.....	15

第 4 章 实验测试.....	16
4.1 测试说明.....	16
4.1.1 测试目标与评价指标.....	16
4.1.2 输入数据、基线与实验控制.....	16
4.1.3 重复评测与统计口径.....	17
4.1.4 功能与平台测试方法.....	17
4.1.5 复现步骤与输出核查.....	18
4.2 测试结果.....	19
4.2.1 最终方案在固定验证集上的性能.....	19
4.2.2 计算量对照与搜索收益.....	20
4.2.3 结构泛化反例与宽并行专项修复.....	20
4.2.4 大规模容量安全与运行开销.....	21
4.2.5 小规模精确解参照.....	22
4.2.6 功能测试与操作系统验证.....	22
4.2.7 结论适用范围.....	23
第 5 章 实现难点说明.....	25
5.1 调度物理一致性与合法性保障.....	25
5.2 策略塌缩与异构动作学习.....	25
5.3 搜索质量、拓扑约束与计算代价平衡.....	25
5.4 宽并行结构与固定模型容量.....	26
5.5 随机实验复现与跨平台证据边界.....	26
第 6 章 总 结.....	27
6.1 完成情况与主要收获.....	27
6.2 适用边界与后续工作.....	27
6.3 开源组件与知识来源.....	27
参 考.....	28

第 1 章 绪论

1.1 目的与意义

本项目面向云、边、端协同执行的静态批任务调度。一次应用由具有先后依赖的有向无环图（DAG）表示，节点携带计算量，边携带前驱输出的数据量；资源具有不同算力和带宽。调度器需要同时确定任务次序、执行资源和起止时间，在满足依赖、通信就绪与资源互斥约束的前提下，缩短全部任务的整体完成时间（makespan）。典型应用包括边缘数据预处理与云端汇总、批量推理流水线、科学计算工作流和构建测试流水线。

设计目的包括三个层面：建立资源和任务参数可配置、策略接口统一的调度框架；探索以启发式先验约束深度强化学习的方法，提升固定验证协议下的调度质量；形成可运行、可追踪、可复核的工程交付。项目将训练、推理、评测和诊断分开，使后续替换策略时可以沿用同一物理模型、合法性检查和统计方法，减少算法迭代伴随实验口径变化的问题。

工程价值在于把“得到一个较小的数值”转化为完整调度计划。输出逐任务资源映射和时间段，提供机器可读结果、单场景命令行、质量档位、时间预算及复现入口；评审者能够从文档定位到原始结果，再检查产生结果的实现。当前系统提供的是离线调度求解与仿真验证能力，实际集群中的任务启动、监控、失败恢复和资源遥测仍需由执行器与运维层接入。

1.2 项目背景

异构资源调度的困难来自多个因素的耦合。算力更强的云资源可能减少执行时长，却会引入数据传输；一条关键依赖链可能限制整体并行度；某些任务虽然已经就绪，仍需等待目标资源上的空闲区间。因此，仅按计算量排序或将所有任务集中到最快资源，通常无法兼顾通信和资源竞争。随着任务数增加，合法拓扑序与资源映射的组合规模迅速扩大，求解质量和决策耗时需要共同考虑。

Topcuoglu 等（2002）提出的 HEFT 以 upward rank 确定优先次序，并使用最早完成时间规则放置任务，具有较低开销和可解释性，适合作为强基线。本项目复用其思想并自主实现统一物理规则下的比较。HEFT 的固定优先级并不保证每个实例全局最优；MILP 可在小规模场景给出精确解或界，但规模扩大后求解与最优性证明成本上升。项目保留两类方法，分别用于快速参照和有限范围的理论核验。

Schulman 等（2017）提出的 PPO 提供了可反复采样和更新的策略优化框架，但通用算法本身并不解决 DAG 依赖是否合法的问题。Huang 与 Ontañón（2020）分析了策略梯度

中的非法动作屏蔽。本项目使用 **MaskablePPO**，只允许从当前就绪任务中采样动作，再通过统一 **EFT** 规则分配资源。训练过程中还需控制观测尺度、奖励尺度和动作语义，避免策略塌缩或将环境差异误当作学习收益。

本项目的特点是把学习、启发式与合法搜索组合为一个可审计求解过程：**HEFT** 提供先验，神经网络学习有界排序残差，多次采样形成不同初解，单任务重定位和大邻域搜索在约束内精修，**Adaptive** 候选集按同一 **makespan** 选优。相关基础方法已有研究积累，本文重点贡献是它们在当前 **DAG** 模型中的实现组合、容量安全与实验闭环；不将 **PPO**、**HEFT** 或 **LNS** 本身表述为项目原创算法。

第2章 技术理论

2.1 设计思想

本项目采用“可验证的物理调度规则+可学习的任务排序+约束搜索精修”的分层设计。学习器负责发现实例相关的任务优先次序，统一的最早完成时间（EFT）规则负责资源放置，候选择优与合法性校验负责最终交付。这样既保留强化学习适应图结构和资源状态的能力，也使每个任务的资源、开始时间和完成时间都能追溯到明确的计算规则。

2.1.1 任务与资源的数学建模

将 workflow 表示为有向无环图 $G=(V,E)$ 。节点 i 的计算量为 w_i ，边 (i,j) 的数据量为 d_{ij} ；资源 r 的计算能力为 p_r ，带宽为 b_r 。云、边、端通过不同的资源参数表达异构性，并使用统一的时间尺度。任务执行不可抢占，同一资源上的执行区间不能重叠。当前模型面向已知 DAG 的离线静态规划，通信代价按资源端点瓶颈带宽计算，不单独模拟链路拥塞和传输队列。

$$t_{i,r} = \frac{w_i}{p_r}$$

任务 i 与 j 位于同一资源时通信时间为零；位于不同资源 r, q 时，通信时间为 $d_{ij}/\min(b_r, b_q)$ 。因此最强算力资源不一定产生最短总完成时间：把全部任务放到云节点可能增加排队，而把关联任务拆到不同节点又可能增加通信等待。系统优化的对象是整个 DAG 的完成时间，而非某个任务的局部执行速度。

$$s_j \geq \max_{i \in \text{pred}(j)} (f_i + c_{ij}(x_i, x_j)), \quad f_j = s_j + t_{j,x_j}$$
$$\min_{x,s} C_{\max}, \quad C_{\max} = \max_{i \in V} f_i$$

其中 x_i 表示任务资源映射， s_i, f_i 分别为开始和完成时间， $\text{pred}(j)$ 表示 j 的前驱集合。源任务的依赖就绪时间取零。代码分别由 `src/env/dag_generator.py`、`src/env/resource_config.py` 和 `src/env/scheduling_utils.py` 实现图结构、执行与通信代价、资源事件时间线。

2.1.2 以统一物理规则保障公平比较

训练环境、HEFT 基线和搜索候选共用 `find_earliest_slot` 插入函数。给定任务的依赖就绪时间后，函数按开始时间扫描资源既有事件，把任务放入满足依赖和持续时间要求的最早空隙；资源选择比较所有资源上的 EFT。若训练环境只把任务追加到时间线尾部，而 HEFT 可以插入空隙，两者即使执行相同顺序也会得到不同结果，实验结论就无法归因于策略。本项目通过统一插入规则 and 同序列重放检查消除这类偏差。

HEFT 的理论依据来自 Topcuoglu 等(2002)。其 upward rank 表示当前任务到出口任务的平均剩余代价，递推式如下；平均计算与通信代价均由本项目的同一资源模型得到。HEFT 根据该优先级形成任务顺序，再用 EFT 分配资源。本项目重新实现这一标准算法，将其同时用于基线、策略先验和安全候选。

$$\text{rank}_u(i) = \bar{t}_i + \max_{j \in \text{succ}(i)} (\bar{c}_{ij} + \text{rank}_u(j))$$

2.1.3 学习与约束的职责划分

最终学习动作只选择当前就绪任务，资源映射由 EFT 确定。该划分把动作规模从“就绪任务数×资源数”降为“就绪任务数”，减少排序与映射相互耦合导致的探索难度。代价是资源选择受启发式规则限制，因此本项目不声称覆盖全部可能的资源映射空间。学习策略在 HEFT 排序基准上预测有界残差，推理再用多样候选和搜索补足单次贪心的局限。

表 2-1 分层职责与正确性保障

层次	主要职责	保障机制
物理调度层	执行、通信、资源放置 makespan	统一代价公式和插入式 EFT
学习构造层	在就绪任务中选择优先顺序	task-only 动作与非法动作屏蔽
约束搜索层	修改合法拓扑序并精修候选	全边拓扑检查、Kahn 修复、严格改善接受
自适应选优层	组合学习与启发式候选	保留精确 HEFT 锚点，按同一 makespan 选优
输出验收层	返回完整可核验调度	任务齐全、时间合法、资源无重叠、依赖满足

系统将“搜索不劣化输入解”和“Adaptive 不劣于 HEFT”分开表述。前者来自 best-only 接受准则，后者来自候选集中始终保留精确 HEFT 锚点。二者是同一模拟器下的设计性质；实验统计用于说明改善幅度和稳定性，不用于替代正确性证明，也不意味着获得全局最优解。

2.2 技术路线

完整流程先载入 DAG 与资源配置，计算图结构和 HEFT 参照；在模型容量内生成 Residual Best-of-64 初解，依次执行单任务重定位和大邻域搜索（LNS）；Adaptive 模式再加入 HEFT 及结构感知候选，并输出最优合法方案。所有候选评估均重新执行统一 EFT 调度。

最终方案：学习构造器 + 约束搜索精修器



只接受严格改善候选：同一初解上 makespan 单调不增

图 2-1 最终调度流程概要；EFT 贯穿每次构造与候选评估，单任务重定位在初解阶段和 LNS 修复阶段执行

2.2.1 环境状态、就绪集合与动作屏蔽

环境按 Gymnasium 接口提供 `reset`、`step` 和字典型观测。观测包含节点计算量、后继数量、后继计算量等任务特征，填充有效性标记、邻接矩阵、就绪槽位到节点编号的映射，以及资源算力、带宽、可用时间和全局进度。连续特征按保存的字段均值和标准差归一化，有效性标记与填充位置保持明确，训练和推理读取同一份 `normalization_stats.json`。Residual 环境额外提供就绪任务的标准化 upward rank。

策略输出对应的是就绪槽位，而不是直接把任务编号当作动作。`action_masks()` 将实际就绪槽位标为合法，未占用槽位标为非法；`step` 再次检查动作范围及就绪条件。训练与推理都把 `mask` 传入策略分布，防止非法动作通过高打分进入候选。Huang 与 Ontañón(2020)讨论了策略梯度中的非法动作屏蔽，本项目通过 SB3-Contrib 的 MaskablePPO 接口落地该机制。

2.2.2 HEFT 残差策略与 MaskablePPO 训练

自定义 `ResidualRankingPolicy` 继承 `MaskableMultiInputActorCriticPolicy`。其任务 logits 由标准化 HEFT rank 与网络残差相加形成，残差通过 `tanh` 限幅。 α 、 β 分别控制启发式先验和学习修正的强度，最终配置均为 1.0。

$$z_i = \alpha \tilde{r}_i + \beta \tanh(g_\theta(o_t, i))$$

残差输出层的权重和偏置全部零初始化，因此初始 logits 只由 HEFT rank 决定。初始确定性动作选择锚定 HEFT 的就绪任务优先顺序；随机采样仍会在合法任务之间形成概率

分布，不能将其理解为每条初始随机轨迹都等价于 HEFT。forward、evaluate_actions 和 get_distribution 使用同一残差公式及 mask，保证采样、训练目标与推理动作分布一致。

训练采用 Schulman 等(2017)提出的 PPO 裁剪目标，并调用 Raffin 等(2021)所述 Stable-Baselines3 框架及 SB3-Contrib 扩展。PPO 通过限制新旧策略概率比的更新幅度，提高多轮小批次训练的稳定性；本项目自行设计调度环境、观测、奖励与残差策略，并通过第三方接口完成 rollout 收集、优势估计、优化和模型保存。

$$L^{\text{CLIP}}(\theta) = E_t[\min(q_t(\theta)\hat{A}_t, \text{clip}(q_t(\theta), 1 - \varepsilon, 1 + \varepsilon)\hat{A}_t)]$$

这里 $q_t(\theta)$ 为新旧策略对同一合法动作的概率比， $\hat{A}_t$ 为优势估计， ε 为裁剪范围。最终训练分布为 8~25 个任务、边密度 0.2~0.5，固定输入容量 max_tasks_padding=30；训练步数 200000，学习率 0.0003，rollout 步数 2048，batch size=64，n_epochs=10， $\gamma = 0.99$ ，GAE $\lambda=0.95$ ，clip_range=0.2，熵系数 0.01，MLP 隐层为 [128,128]，训练种子为 123。权威配置保存在 training/configs/ppo_mlp_residual.yaml。

2.2.3 相对 HEFT 的终局奖励

relative_heft 模式每步给出 -0.01 的小惩罚，只有完成全部任务时再加入相对 HEFT 的性能差距。令当前完整方案完成时间为 C、同场景 HEFT 完成时间为 C^H ，则有效参照条件下的奖励为：

$$r_t = -0.01, \quad r_T = -0.01 - \left(\frac{C}{C^H} - 1\right)$$

相对比值消除了不同任务规模和计算量造成的绝对时间尺度差异，同场景优于 HEFT 时终局增加收益，劣于 HEFT 时增加惩罚。对固定任务数而言，每步常数不会区分同一实例的任务顺序，其作用是控制回报量级；策略质量仍主要由终局相对性能提供信号。

HEFT 参照使用独立资源配置计算，避免被环境调度状态污染。

2.2.4 多样初解与合法搜索精修

不受时间预算截断时，Best-of-64 对同一 DAG 生成 64 条随机合法调度轨迹，另加入一次确定性轨迹，保留 makespan 最小的任务序列。软预算模式下实际采样数可能减少，截止后也可能跳过确定性轨迹。多次采样把策略随机性转化为候选多样性，但推理成本也随采样数增加。项目提供 fast、balanced、quality 三档，分别使用“采样数/重定位轮数/LNS 次数”为 8/1/8、32/2/32、64/3/64，便于按现场时间预算选择。

单任务重定位枚举将一个任务移到另一个位置后的序列。实际代码检查每条 DAG 边两端的位置，拒绝任何前驱位于后继之后的候选；合法候选重新进行 EFT 重放，每轮选

择改善最大的移动，达到轮次上限或没有改善时停止。此过程改变的是任务优先序，资源映射和时间线由重放重新计算，不能沿用原序列中已经过期的时间。

LNS 借鉴 Ropke 与 Pisinger(2006)所述破坏—修复搜索思想。本项目针对 DAG 重新实现算子：随机选择 2~4 个任务，释放其原位置优先级；其他任务保留原位置锚点；再用带优先队列的 Kahn 算法从当前零入度集合重建拓扑序，并执行一轮重定位。接受条件为候选 makespan 比当前最优值至少小 10^{-12} ；相等或变差候选被拒绝。该实现没有使用按算子历史收益更新权重的经典 ALNS 机制。

$$C_{k+1} \leq C_k$$

上述单调性质来自始终保留 incumbent 和严格改善接受，适用于同一初解、同一执行代价和浮点容差。它不保证搜索能越过需要暂时变差的局部障碍，因此仍需多种初解与随机破坏来扩大覆盖范围。src/policies/residual_local_search_scheduler.py 同时记录评估邻居数、接受移动数、完成迭代数和超时状态，使性能与计算投入可以对应。

2.2.5 Adaptive 候选组合、容量安全与软预算

Adaptive 首先计算任务数、边数、源汇节点数、层宽、最长路径及边密度。模型容量内可使用 Residual+搜索候选；所有图均保留精确 HEFT 锚点，并在预算允许时加入 HEFT+LNS。宽度比例不小于 0.30 时增加宽度感知和成本优先拓扑序，两者仍仅从就绪集合选任务，并分别使用 rank、计算量和出度构造不同优先级，随后接受相同搜索精修。

最终选择所有候选中 makespan 最小的方案。设候选集 $\mathbb{C}$ 包含 HEFT 调度 S^H ，则由选优定义直接得到下式。该保证依赖相同资源模型和调度模拟器，不要求神经策略在每个实例上优于 HEFT。

$$S^* = \arg \min_{S \in \mathbb{C}} C_{\max}(S), S^H \in \mathbb{C} \Rightarrow C_{\max}(S^*) \leq C_{\max}(S^H)$$

最终 checkpoint 的输入容量为 30 个任务。图规模超过 30 时，Adaptive 记录明确的容量超限原因，跳过 Residual 候选，继续 HEFT/LNS 路径；residual_only 模式则明确报错。系统不会通过裁剪 DAG 假装完成大图，也不将大规模安全路径的结果归因于神经网络规模外泛化。

墙钟预算在 rollout、重定位候选和 LNS 迭代之间检查，到期后停止新增昂贵计算并返回当前最优方案。已经开始的原子计算及 HEFT 锚点允许完成，因此这是软截止，实际耗时可能略超预算。模型载入与调度阶段分别计时；最终输出包含候选来源、候选性能、超时状态、模型跳过原因和合法性检查结果。

2.2.6 统一接口与实验复现

HEFT、RL、MILP 和 Hybrid 调度器采用 `BaseScheduler.schedule(dag,resource_config)` 接口，返回 `task_id` 到 “`resource_id`、`start_time`、`finish_time`” 的映射。评测器只依赖统一结果结构，独立计算 `makespan` 与相对 HEFT 比值。MILP 使用 PuLP/CBC 表达资源分配、依赖和不重叠约束，只在求解器明确证明最优时作为全局最优参照；超时可行解保留状态及界，不混入“距最优”统计。

复现实验固定场景文件与资源配置，保存配置、checkpoint、采样种子、逐场景 JSON 和汇总结果。五次重复使用同一训练模型，仅变化推理采样与搜索随机性；同轮方法共享采样种子，并对每轮 20 场景均值做配对比较。数据汇总、合法性检查、Windows/Linux 脚本和国产操作系统容器矩阵共同组成可核验的工程闭环。

2.3 代码原创说明

本项目的贡献集中在 DAG 调度环境与统一物理规则、HEFT 先验注入的残差排序策略、合法重定位及 DAG 破坏—修复算子、Adaptive 安全选优和实验审计工具。标准算法思想、第三方库实现和本项目调度逻辑分别列明，第三方依赖通过安装包与公开接口使用，不把依赖源码规模计入本项目原创代码规模。

2.3.1 第三方库及其引用方式

下表版本按提交仓库 `requirements.txt` 声明，PyTorch 按国产操作系统 Dockerfile 单独固定的 CPU 版本列出。版本号表示该项目的依赖配置，实际复现环境仍以保存的依赖快照与运行清单为准。

表 2-2 主要第三方依赖、用途及代码引用方式

库及声明版本	项目用途	引用方式与对应入口
NetworkX 3.4.2	DAG 建模、拓扑排序、图结构验证	<code>import networkx as nx;</code> <code>src/env/dag_generator.py</code> 、HEFT 与结构统计模块
Gymnasium 1.3.0	环境基类与观测、动作空间	<code>import gymnasium;</code> 继承 <code>gym.Env</code> ，调用 <code>spaces.Dict/Box/Discrete</code>
NumPy 2.2.6	观测张量、归一化、随机种子	<code>import numpy as np;</code> <code>src/env/</code> 、 <code>src/policies/</code> 、 <code>src/evaluation/</code>
PyYAML 6.0.3	资源及训练配置加载	<code>import yaml;</code> 调用 <code>safe_load</code> 读取 YAML
PyTorch 2.12.1+cpu（容器）	网络、自动微分、零初始化、随机种子	<code>import torch;</code> <code>ResidualRankingPolicy</code> 中调用张量、 <code>nn.init</code> 与 <code>tanh</code>

库及声明版本	项目用途	引用方式与对应入口
Stable-Baselines3 2.9.0	训练基础设施、回调、监控与策略接口	from stable_baselines3.common... import; 使用 Monitor、DummyVecEnv、BaseCallback 等
SB3-Contrib 2.9.0	MaskablePPO 与可屏蔽策略分布	from sb3_contrib import MaskablePPO; 继承 MaskableMultiInputActorCriticPolicy
PuLP 3.3.2 / CBC	小规模 MILP 参照求解	import pulp; 构建 LpProblem/LpVariable 并调用 CBC 接口
SciPy 1.15.3	配对 t 检验等统计分析	from scipy.stats import ttest_rel; src/evaluation/ 统计脚本
Matplotlib 3.10.9	调度图与结果可视化	import matplotlib.pyplot as plt; scripts/ 可视化脚本
pytest 9.1.1	回归测试与正确性断言	测试函数由 pytest 执行; tests/
TensorBoard 2.21.0	训练指标与事件日志	通过 SB3 tensorboard_log 参数及日志兼容代码调用
tqdm 4.70.0、Rich 15.0.0	训练进度显示配套依赖	由 SB3 progress_bar 接口使用, 不属于调度算法逻辑

第三方来源为各库官方项目及文档: NetworkX (networkx.org)、Gymnasium (gymnasium.farama.org)、NumPy (numpy.org)、PyYAML (pyyaml.org)、PyTorch (pytorch.org)、Stable-Baselines3 与 SB3-Contrib (stable-baselines3.readthedocs.io、sb3-contrib.readthedocs.io)、PuLP/CBC (coin-or.github.io/pulp、github.com/coin-or/Cbc)、SciPy (scipy.org)、Matplotlib (matplotlib.org)、pytest (docs.pytest.org)、TensorBoard (github.com/tensorflow/tensorboard)、tqdm (tqdm.github.io)、Rich (rich.readthedocs.io)。具体 API 调用可沿表中模块核对。

2.3.2 算法依据与项目实现的界限

HEFT 的 upward rank 与插入式 EFT 属于 Topcuoglu 等(2002)提出的标准方法, 本项目按资源模型重新实现基线及重放逻辑, 不将算法思想宣称为原创。PPO 和非法动作屏蔽分别依托 Schulman 等(2017)、Huang 与 Ontañón(2020), 优化器和分布基础设施通过 SB3/SB3-Contrib 调用; 本项目在派生策略中实现 rank 加性先验、零初始化残差和统一前向/动作评估逻辑。

LNS 的破坏—修复属于已有搜索范式; 本项目实现适用于 DAG 的位置锚点释放、零入度拓扑修复与严格改善接受。宽度感知候选组合、容量超限跳过、软预算和 HEFT 锚点

非回归规则由本项目代码整合，不等同于论文中的车辆路径 ALNS 算子。复现与审计模块组织本项目生成的场景、日志和结果，并保留失败实验作为方法取舍依据。

表 2-3 项目自定义模块及其来源边界

模块	提交仓库中的实现内容	来源说明
src/env/	DAG、资源配置、调度环境、归一化、统一插入函数	面向项目需求编写；图与环境接口使用第三方库
src/baselines/heft_scheduler.py	upward rank、EFT 与时间线构造	依据公开 HEFT 定义重新实现
src/policies/residual_ranking_policy.py	先验 logits、残差限幅、输出层零初始化	项目策略适配代码；继承第三方策略基类
src/policies/residual_local_search_scheduler.py	重定位、DAG LNS、结构候选与安全选优	项目自定义搜索与组合逻辑；借鉴公开 LNS 范式
src/training/、src/evaluation/、scripts/、tests/	配置驱动训练、统计、校验、命令行和复现	项目工程代码；调用公开训练、求解及统计接口

关于第三方源代码的直接复制及改写：本次依据仓库文件、依赖声明和 `import` 关系核对了引用来源与调用方式，但没有完成逐行源码溯源，原样复制或改写的具体行数尚未统计核验。因此此项如实登记为“未完成行级统计”，不将接口调用数量换算为复制行数，也不填写缺乏证据的“完全引用 0 行”或原创比例。依赖安装目录中的第三方源码不属于本项目提交的原创模块。

第 3 章 项目介绍

3.1 功能介绍

项目按环境与资源、基线、学习策略、训练、评测诊断和工程入口划分模块。主要数据流为“场景 JSON 与资源 YAML→构造合法任务顺序→EFT 生成时间线→搜索精修→调度合法性校验→结果 JSON 及可视化”。训练和推理共用环境定义与调度物理规则；报告生成读取已有证据文件，不回写实验结果。各模块职责和接口见下表。

表 3-1 主要功能模块与实现入口

模块	主要职责	实现入口
环境与资源	DAG 生成/加载、就绪集合、观测与资源时间线	src/env/、configs/
基线与统一接口	HEFT、MILP、Hybrid 及 makespan 计算	src/baselines/、 src/scheduler_interface.py
学习与搜索策略	Residual 排序、多样化采样、重定位、LNS 和候选择优	src/policies/
训练与配置	MaskablePPO 训练、checkpoint、日志和参数文件	src/training/（源码）； training/（配置、模型或数据）
评测、诊断与交付	固定场景、配对统计、合法性、审计、CLI 和复现	src/evaluation/、evaluation/（数据）、src/diagnostics/、scripts/

3.1.1 任务与资源配置

场景采用 NetworkX node-link JSON，节点必须具有正的 `computation_cost`，依赖边使用非负 `data_size`；加载器使用 links 保存边并统一整数任务编号。资源 YAML 列出 id、tier、compute_power 和 bandwidth，可更换资源数量及异构强度。默认配置包含 1 个云、2 个边和 1 个端资源，算力依次为 100、30、25、5，带宽依次为 1000、200、150、20。任务执行时间按计算量与资源算力之比估计，跨资源通信按两端较小带宽估计，同资源通信为零。

输入检查拒绝空图、循环依赖、缺少计算量、非正计算量和负数据量；资源标识必须唯一。配置和检查让用户能够调整实验，而无需修改调度算法。参数变化后应重新评测，默认四资源结果不能直接外推到新的网络和硬件配置。

3.1.2 统一调度接口与合法输出

所有调度器实现 `BaseScheduler.schedule(dag, resource_config)`，返回 `task_id`→`(resource_id, start_time, finish_time)`。`compute_makespan` 统一取最大完成时间。该契约使

HEFT、MILP、Hybrid、RL 和搜索方案可以在同一评测器中替换。对资源副本独立调度，避免前一方法留下的时间线影响后一方法。

单场景入口 `scripts/schedule.py` 支持 `adaptive` 和 `heft`。执行结束后逐项检查任务是否齐全、资源是否合法、时间是否非负、同资源是否重叠以及每条依赖是否满足通信就绪时间。无效计划触发错误，不能仅凭较小 `makespan` 进入交付。输出包含输入来源、参数、DAG 规模、结果、模型加载和求解耗时、进程内存、候选诊断、校验结论及完整任务计划。

3.1.3 可训练策略与质量控制

训练入口 `src/training/train_ppo.py` 加载配置，组织环境、MaskablePPO、日志和模型保存。最终推理使用 Residual 模型作为候选构造器，并保留 HEFT 锚点。搜索只接受严格缩短 `makespan` 的候选；超出模型 30 任务输入容量时记录跳过原因并启用 HEFT/LNS 安全路径。能够继续输出有效调度，不能据此声称神经网络已学会任意规模泛化。

表 3-2 单场景 CLI 质量档位

档位	采样数	重定位轮数	LNS 迭代数	使用目的
fast	8	1	8	快速检查和轻量演示
balanced	32	2	32	默认运行，兼顾耗时与质量
quality	64	3	64	较充分搜索与质量展示

用户可通过 `--num-samples`、`--local-max-passes` 和 `--lns-iterations` 覆盖档位，或用 `--time-budget-seconds` 设置软墙钟预算。预算检查发生在候选之间，原子搜索不会被中途强制终止，因此可能略超设置时间。更高档位增加搜索机会，也增加时间成本；最终耗时取决于图规模、形状和机器负载。

3.1.4 评测与诊断

`evaluation` 提供固定场景比较、结构泛化、规模测试、MILP 核验及配对统计；`diagnostics` 检查观测、动作屏蔽、梯度、报告数字和结构。测试覆盖物理时间线、奖励、策略残差、拓扑合法性、候选非回归、容量降级和输入输出。正式结论采用固定协议下的重复结果；单次 CLI 数值用于核对接口，不能替代统计均值。

3.1.5 运行、复现与工程交付

项目提供 Windows 和 Linux 入口，以及 openEuler、openKylin 和 Anolis OS 的 Docker 定义与运行证据。推荐 Python 3.10~3.12 虚拟环境，按 requirements.txt 安装直接依赖；CPU 推理可运行。scripts/reproduce.py 提供 smoke、wide 和 final 档位，分别用于编译测试与场景检查、宽并行快速复核（默认 16 次采样/16 轮 LNS；正式 64/64 结果需显式指定参数）、正式五次重复统计。运行者应保留所用模型、配置、随机种子、原始 JSON 和清单，便于识别证据版本。

交付材料包含源码、配置、checkpoint、固定评测场景、说明书、演示 PPT、视频及原创承诺书。文档以仓库内相对路径定位证据，避免暴露本机身份。当前容器结果证明相应发行版用户态与 Python 依赖栈可运行；Windows Docker Desktop 共享 WSL2 内核，不能写成三种系统物理机原生内核性能验证。

3.2 软件界面

软件的主要操作界面是命令行与结果文件，界面设计遵循“输入明确、结果可读、约束可见、证据可定位”。命令行接受场景、资源、模型、策略、质量档位、种子和时间预算；终端摘要帮助快速判断运行是否完成；JSON 保存完整计划供程序消费；DAG 与甘特图用于检查任务依赖和资源时间线。

3.2.1 命令行调度操作

以下示例在项目根目录以 Bash 运行，使用已有 10 任务场景、默认资源、balanced 档位和 10 秒预算；Windows 可将参数写在同一行。默认模型与训练配置由脚本填入。--method heft 可生成同场景 HEFT 参考；--output 仅将 JSON 写到标准输出，便于脚本集成。保存到文件时，成功运行打印 SCHEDULE_COMPLETE 及候选来源、任务数、makespan、ratio、求解耗时、合法性和输出位置。

```
python scripts/schedule.py \
    --input evaluation/scenarios/scenario_10_0.json \
    --resources configs/resource_default.yaml \
    --method adaptive --preset balanced \
    --time-budget-seconds 10 --seed 20260905 \
    --output evaluation/results/schedule_output.json
```

3.2.2 机器可读结果与资源甘特图

evaluation/results/schedule_output.json 保存的实例具有 10 个任务、16 条依赖。其 selected_source 为 residual_ins，makespan 为 0.390745507260，HEFT 为 0.455228436345，ratio_to_heft 为 0.858350393040，求解耗时约 1.299 秒，validation.valid 为 true。相对 HEFT 缩短约 14.16%是该实例的结果，未纳入正式重复统计；图中时间轴表示模型中的调度时间，1.299 秒表示求解器实际墙钟耗时，二者含义不同。

表 3-3 结果 JSON 的主要字段与阅读方法

字段	记录内容	阅读用途
input、parameters、dag	场景及配置路径、档位、种子、预算、图规模	确认同一次运行的条件
result、timing、memory	makespan、HEFT 比值、分阶段耗时、RSS	分别核对调度质量与计算成本
selected_source、diagnostics	选中候选、各候选质量与耗时、降级或超时信息	说明选优及容量安全路径
validation、schedule	合法性结论与逐任务资源、起止时间	检查输出并对接后续执行器

scripts/visualize_schedule.py 读取原始场景和上述 JSON 生成图像，左侧显示 DAG 及最终资源归属，右侧按云、边、端泳道绘制任务时间段，并标记本方法和 HEFT 的整体完成时间。图像不替代校验器，但能直观发现空闲区间、并行执行和结束时间差异；可视化必须与对应 JSON 同步，不能用手绘结果冒充实测。

云一边一端异构计算资源调度结果

由真实场景与调度结果文件即时生成 | method=adaptive

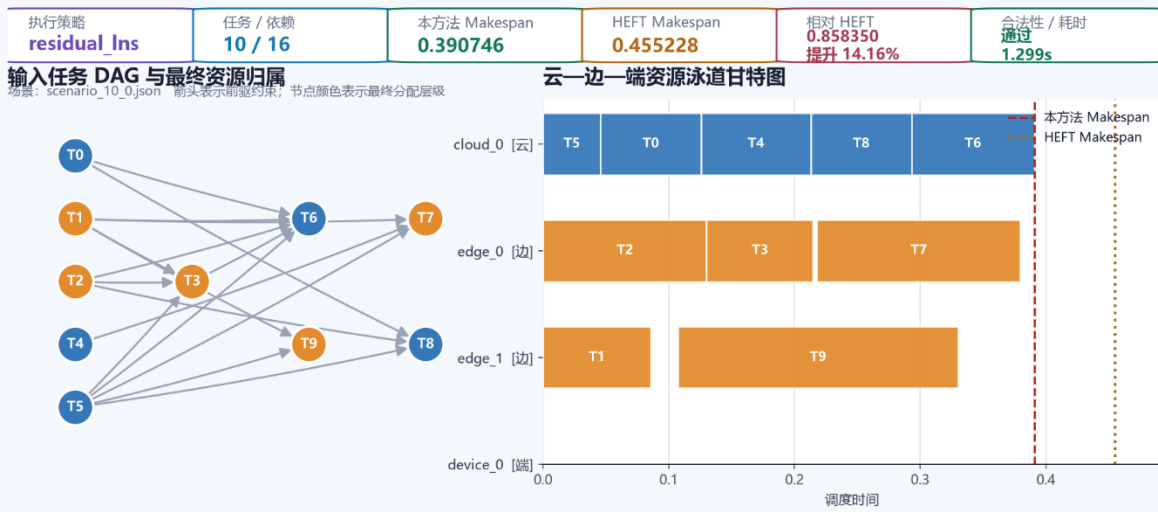


图 3-1 真实 10 任务实例的 DAG 与资源甘特图（读取 scenario_10_0.json 与 schedule_output.json 生成；单场景示例）

3.2.3 本地演示附件的范围

本地 artifacts/video_recording_v3 中另有浏览器演示附件，用于展示命令运行、指标和调度可视化；它尚属本地未追踪材料，不作为已部署远程 Web 服务的交付声明。本说明书采用仓库内真实 CLI、JSON 和生成图作为正式界面证据，避免将录屏工具画面或本机隐私窗口作为软件功能截图。

第 4 章 实验测试

4.1 测试说明

4.1.1 测试目标与评价指标

测试围绕调度质量、约束正确性、容量边界和操作系统兼容性展开。性能测试回答最终组合能否缩短 DAG 的整体完成时间；功能测试检查任务覆盖、前驱依赖、通信等待、资源互斥和输出完整性；规模测试观察超过模型容量后的降级路径与运行成本。各项结果均绑定到仓库中的原始 JSON、测试文件或运行日志，正式统计与单场景演示分别说明。

主指标为 `mean_ratio`：先计算每个场景的被测方法 `makespan` 与 HEFT `makespan` 之比，再对场景取算术平均。HEFT 对应 1，数值越小越好，相对 HEFT 的平均缩短比例为 $(1 - \text{mean_ratio}) \times 100\%$ 。这种口径避免用不同规模场景的绝对时间直接求平均。`makespan` 表示模型中的任务完成时间；`schedule_seconds` 表示算法在计算机上生成方案的墙钟耗时，二者含义不同。

4.1.2 输入数据、基线与实验控制

最终模型配置见 `training/configs/ppo_mlp_residual.yaml`，训练任务规模为 8~25，边密度配置为 0.2~0.5，输入 `padding` 容量为 30，训练步数配置为 200000，训练种子为 123。主验证集是 `evaluation/scenarios/` 中固定保存的 20 个 DAG，包含 10、15、20、25 任务各 5 个，生成种子从 1000000 开始。训练采用独立生成序列，评测读取持久化输入，不在每轮更换 DAG。

所有方法使用 `configs/resource_default.yaml` 中相同的一个云节点、两个边节点和一个端节点，以及相同的执行时间、通信时延和插入式资源时间线。每次调度重新加载资源状态，防止前一方法的可用时间污染后续结果。HEFT 是主要基线；Residual Best-of-64 用于隔离搜索与候选组合的收益；Best-of-128 是历史计算量对照；MILP 只在求解器证明最优的小规模实例上充当理论参照。

表 4-1 各项实验的输入与用途

实验	输入规模	被测对象	主要用途
主性能评测	10/15/20/25 各 5 个	同一 Residual 模型的最终 Adaptive	固定验证集性能与采样稳定性
结构泛化	宽并行、深链、同构、对照各 5 个	历史 Residual Best-of-64	识别训练分布的结构短板
宽并行专项	新版本宽并行 DAG 共 5 个	Adaptive, 64 次采样/64 轮 LNS	检查候选机制的专项效果

实验	输入规模	被测对象	主要用途
规模与开销	30/40/50/60 各 5 个	完整 Adaptive 系统	容量降级、时间与内存
精确解参照	10/15 任务共 10 个	HEFT 与历史 Best-of-16	估计已证明最优实例上的差距

结构泛化证据见 [evaluation/results/structural_generalization/](#)，宽并行专项输入见 [evaluation/scenarios_structural_v2/wide_parallel/](#)，大规模输入见 [evaluation/scenarios_large/](#)。这些集合的任务结构、资源条件或预算不同，分别回答迁移能力和工程扩展问题，不将其均值与主验证集均值直接排名。

4.1.3 重复评测与统计口径

正式主结果使用同一个 `training/checkpoints/ppo_mlp_residual.zip`，进行五次完整推理评测。五个规范采样种子为 1565812275、842234145、386081360、1117038938、1760006972，最初由系统熵产生，复现时固定保存。每轮对第 j 个固定场景使用该轮采样种子加 j ，并同步设置 Python、NumPy 与 PyTorch 随机状态；原始 Residual 与最终搜索阶段共用该场景的配对初始结果。

五次重复改变的是推理采样随机性，并非五次独立训练；因此稳定性结论限定于当前 checkpoint、固定场景及这五个种子。主结果的“均值 $\pm$ 标准差”以五轮完整评测的均值为样本， $\pm$ 为样本标准差，不是标准误或置信区间。大规模和专项结果的标准差以不同场景为样本，统计单位与主结果不同。

配对检验以每轮最终 `mean_ratio` 减同轮基线 `mean_ratio` 为差值，采用双侧 t 检验，显著性阈值为 0.05。解释同时考虑差值方向、改善幅度和反超场景数， p 值不代表收益大小。五轮数据全部保留在 [evaluation/results/final_pipeline_ins_repeats/](#)，汇总为 [evaluation/results/final_pipeline_ins_summary.json](#)，不用最好一次代替总体统计。

4.1.4 功能与平台测试方法

功能测试覆盖构造、搜索和交付三个环节。环境测试检查观测、`ready` 列表及合法动作 `mask`；一致性测试将相同任务分配重放到环境，校验与 HEFT 的 `makespan` 一致；搜索测试检查合法拓扑序和 `best-only` 接受规则；接口测试检查完整 JSON 与非法重叠的拒绝；边界测试检查预算耗尽后仍返回合法方案，以及超出 30 任务时跳过模型。

运行时验收由 `src/evaluation/schedule_validation.py` 执行：检查是否遗漏或增加任务、资源标识是否存在、开始时间是否非负、结束时间是否早于开始时间；按资源排序区间后检查重叠，并对每条依赖验证后继开始时间不早于前驱完成时间加通信延迟。默认浮点容

差为 10^{-9} 。验收结果同时保存预期任务数、已调度任务数和错误列表，使低 makespan 必须与完整合法输出共同成立。

表 4-2 关键功能的验收依据

检查对象	验收条件	测试文件
时间线一致性	相同分配重放的 makespan 一致	tests/test_scheduling_consistency.py
重定位与 LNS	保持前驱关系且不劣化当前方案	tests/test_residual_local_search_scheduler.py
系统输出	任务完整、JSON 可序列化、拒绝资源重叠	tests/test_schedule_cli.py
容量与预算边界	超容量跳过模型；截止后合法且不弱于锚点	tests/test_portfolio_robustness.py

操作系统测试使用 openEuler 24.03 LTS-SP4、openKylin 2.0 SP1、Anolis OS 23 的 Docker 用户态环境，执行身份记录、依赖检查、字节码编译、pytest 和场景生成。历史三系统矩阵与后续演示记录分别保留，避免把新增测试数量回填到旧日志。容器共享 WSL2 宿主内核，因此该测试验证发行版用户态及 Python 依赖兼容性，不等同于三种原生内核或物理设备验证。

4.1.5 复现步骤与输出核查

复现时在仓库根目录按 requirements.txt 安装依赖并使用已有 checkpoint。建议在仓库副本中运行，以保留交付的正式证据。Windows 和 Linux 一键脚本会检查依赖及模型、准备固定场景、依次执行五个规范种子并生成配对统计；checkpoint 缺失时脚本会进入训练流程，此时新模型的数值不应直接视为原 checkpoint 的复现结果。

```
# Windows PowerShell
powershell -ExecutionPolicy Bypass -File scripts/run_final_pipeline.ps1

# Linux / openEuler
bash scripts/run_final_pipeline.sh

# 功能测试
python -m pytest tests -q

# 三系统矩阵
python scripts/run_os_matrix.py --pull --strict
```

大规模复现使用同一批持久化输入。质量档采用 64 次采样、3 轮重定位、64 轮 LNS，每场景 10 秒软墙钟预算；固定工作量档采用 8/1/8 并关闭截止。软预算在 rollout 与

邻域评估间检查，最后一次评估和必要锚点构造允许完成，不能解释为严格的硬实时上限。

```
python src/evaluation/benchmark_scalability.py --seed 20260905 --num-samples 64 --local-max-passes 3 --lns-iterations 64 --time-budget-seconds 10 --results-path audit_outputs/large_quality.json
```

```
python src/evaluation/benchmark_scalability.py --seed 20260905 --num-samples 8 --local-max-passes 1 --lns-iterations 8 --time-budget-seconds 0 --results-path audit_outputs/large_fixed.json
```

```
python scripts/schedule.py --input evaluation/scenarios/scenario_10_0.json --output audit_outputs/schedule.json --preset quality --time-budget-seconds 8 --seed 20260905
```

核查顺序为输入路径、checkpoint、参数和种子，随后检查 validation.valid、任务数、候选来源、makespan 比值与统计量。墙钟耗时受机器负载影响，跨版本和跨处理器的随机采样浮点结果也可能存在差异。固定工作量更适合比较确定性流程与耗时趋势；预算档用于评估实际延迟约束下的质量，不以不同机器的秒数作未经控制的速度排名。

4.2 测试结果

4.2.1 最终方案在固定验证集上的性能

最终 Adaptive portfolio、合法重定位与 LNS 的五轮 mean_ratio 为 0.916441 ± 0.000984 ，相对 HEFT 平均缩短 makespan 8.3559%；每轮均有 20/20 个场景严格优于 HEFT。五轮均值范围为 $0.915591 \sim 0.917625$ ，表明在该模型和固定场景中，候选组合与搜索降低了对 rollout 随机性的敏感度。

表 4-3 最终方案的五轮配对结果

轮次	Residual 比值	Adaptive 比值	Adaptive 反超数	20 场景耗时/s
1	0.949533	0.917625	20/20	123.950
2	0.948910	0.915701	20/20	121.122
3	0.956776	0.915895	20/20	122.066
4	0.948961	0.915591	20/20	122.377
5	0.949891	0.917393	20/20	129.195

配对原始 Residual Best-of-64 为 0.950814 ± 0.003358 ，平均反超 18.2/20 个场景。最终方案与其比值差为 -0.034373 ± 0.003685 ，即降低 3.4373 个比值百分点；按两组均值计算，相对 Residual 的改善约为 3.6151%。双侧配对检验 $t = -20.859262$ 、自由度 4、 $p = 3.121263 \times 10^{-5}$ 。五轮纯评测合计 618.709 秒，平均每轮 123.742 秒，不含模型训练。证据：evaluation/results/final_pipeline_lns_summary.json。

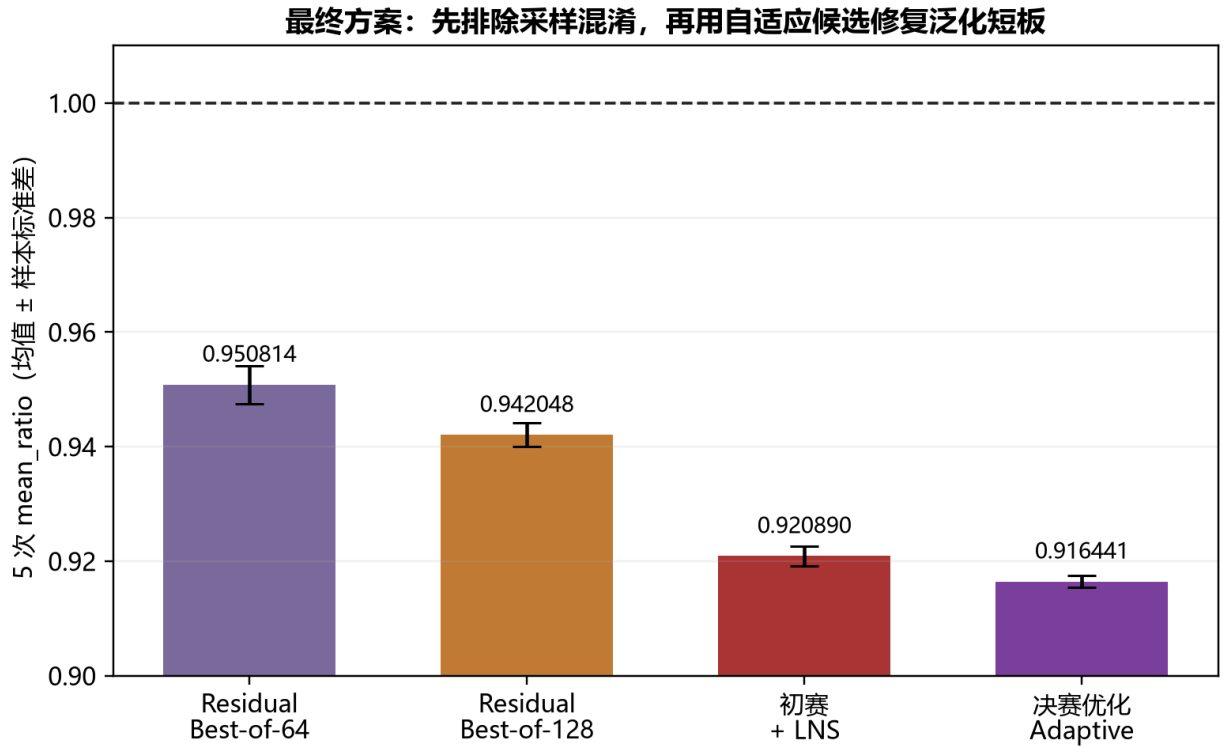


图 4-1 固定验证集的主性能与历史对照。来源：docs/figures/final_statistics.png；柱值及误差线与对应原始汇总一致，误差线为五轮均值的样本标准差。Best-of-128 的计算量对照针对初赛 LNS，不表示与决赛 Adaptive 等耗时。

4.2.2 计算量对照与搜索收益

为区分增加采样次数与搜索结构的作用，历史实验先按墙钟耗时校准纯采样数量，再锁定 Best-of-128。纯采样得到 0.942048 ± 0.002063 ，20 场景平均耗时 56.802 秒；初赛 Residual 加重定位及 LNS 得到 0.920890 ± 0.001729 ，平均耗时 54.402 秒。LNS 与纯采样配对差为 -0.021158 ± 0.001148 ， $p=2.072081 \times 10^{-6}$ 。

纯采样略多耗时仍表现较差，支持当前验证协议下邻域搜索具有独立价值。该对照对应初赛 LNS，决赛 Adaptive 增加多来源候选，运行成本更高，不能将旧对照写成最终方案已在相同预算下击败 Best-of-128。证据：

evaluation/results/autonomous_exploration/compute_matched_sampling/paired_comparison_summary.json。

4.2.3 结构泛化反例与宽并行专项修复

历史结构泛化使用原始 Residual Best-of-64。宽并行组均值超过 1，仅 2/5 反超 HEFT，说明短链、多就绪任务的图结构是原模型的短板；深链条和同构资源组则优于

HEFT。该反例完整保留，为决赛增加 HEFT 锚点、宽度感知顺序和代价优先候选提供依据。

表 4-4 历史 Residual 的结构泛化结果

场景组	mean_ratio	场景标准差	反超 HEFT
宽并行	1.011915	0.018469	2/5
深链条	0.943597	0.043357	4/5
同构资源	0.879572	0.071489	5/5
原始分布对照	0.964085	0.040774	4/5

表中数据来自

evaluation/results/structural_generalization/summary_structural_generalization.json。决赛专项在新版本宽并行集上，以同一 checkpoint、种子 20260830、64 次采样和 64 轮 LNS 取得 mean_ratio=0.929848；4/5 严格优于 HEFT，全部 5/5 不劣于 HEFT。打平的 scenario_8_0.json 经 MILP 证明最优，最优 makespan 为 0.330808745，求解约 0.442 秒。

同批 residual_only 搜索路径均值为 0.933210，Adaptive 相对它改善 3/5、其余持平，平均差值 -0.003362。证据：evaluation/results/wide_parallel_adaptive.json。由于历史与新版专项输入不同，1.011915 到 0.929848 用于说明诊断和修复闭环，不作为同一组实例的配对效应。5 个场景的专项结果也不外推为任意宽并行 DAG 的普遍收益。

4.2.4 大规模容量安全与运行开销

固定 20 个大规模场景上，10 秒质量档 mean_ratio 为 0.941627 ± 0.026569 ，平均缩短 5.8373%，20/20 严格优于 HEFT，95% 均值置信区间为 [0.929192, 0.954062]，检验 $p=6.985758 \times 10^{-9}$ 。固定工作量档为 0.949723 ± 0.023441 ，缩短 5.0277%，同样 20/20 反超， $p=1.025617 \times 10^{-8}$ 。

表 4-5 30~60 任务的质量与资源开销

协议	mean_ratio ± 标准差	反超 数	平均/P95 耗时/s	峰值 RSS/MiB
64/3/64, 10 秒 软预算	0.941627 ± 0.026569	20/20	10.002 / 10.003	358.1
8/1/8, 无截止	0.949723 ± 0.023441	20/20	6.633 / 13.284	359.1

表 4-6 不同任务规模的质量与固定工作量耗时

任务数	场景数	10 秒档比值	固定工作量 比值	固定档平均耗时 /s
30	5	0.934907	0.939627	3.641
40	5	0.926731	0.941565	3.112
50	5	0.948592	0.957592	6.590
60	5	0.956278	0.960109	13.188

质量档全部场景触达软预算，平均进程累计 CPU 时间为 21.259 秒，平均墙钟时间约 10 秒；CPU 时间包含进程线程累计执行，不能与墙钟时间混用。固定工作量平均 CPU 时间为 8.705 秒。RSS 是调度进程内存，包含模型和依赖，不等于新增搜索内存。质量档平均耗时约为 HEFT 的 2118.8 倍，表明系统适合秒级离线批调度或计划生成，当前实现不适合毫秒级硬实时决策。

30 任务仍可使用 Residual 候选；40~60 任务共 15 个场景均明确跳过容量为 30 的模型，最终来源为 heft_ins，且输出通过合法性校验。这里验证的是完整系统的安全扩展，不能据此宣称神经网络在 40~60 任务上完成规模泛化。原始记录：

evaluation/results/final_adaptive_large_scale.json 与
evaluation/results/final_adaptive_large_scale_fixed.json。

4.2.5 小规模精确解参照

MILP 在 10/15 任务共 10 个场景上采用每场景 300 秒求解上限，其中 7 个证明全局最优，3 个仅获得可行解。仅在已证明最优的 7 个场景中，HEFT/Optimal 的均值为 1.106807，历史 Residual Best-of-16/Optimal 为 1.044188，分别高于最优解 10.6807% 和 4.4188%。数据支持学习构造方法在这些小实例上更接近理论最优。

3 个未证明最优的结果只记录 incumbent、best bound 和 gap，不混入“距离最优”的均值。该实验对象是早期 Best-of-16，不能将 4.4188% 写成最终 Adaptive 的最优差距；最终方案尚无覆盖全部验证场景的最优性证明。证据：

evaluation/results/milp_optimal_comparison.json，求解日志见
evaluation/results/milp_solver_logs/。

4.2.6 功能测试与操作系统验证

2026 年 8 月 31 日三系统严格矩阵真实记录全部通过：三个发行版均完成编译、当时完整的 38 项测试，以及 20 个结构场景生成，每套为 38 passed、17 warnings。构建和运行

退出码均为 0，PyTorch 均为 2.12.1+cpu、CUDA 不可用。历史警告主要为 PuLP 接口弃用提示，原始日志按当时软件栈保留。

表 4-7 历史三系统矩阵与最新本地演示验收

日期	系统	Pyth on	实际完成的检查
2026-08-31	openEuler 24.03 LTS-SP4	3.11.6	编译、38 项 pytest、场景生成通过
2026-08-31	openKylin 2.0 SP1	3.12.2	编译、38 项 pytest、场景生成通过
2026-08-31	Anolis OS 23	2 3.10.1	编译、38 项 pytest、场景生成通过
2026-09-22	openEuler 24.03 LTS-SP4	3.11.6	编译通过；44 passed，202 warnings
2026-09-22	openKylin 2.0 SP1	3.12.2	身份、Python、compileall 通过
2026-09-22	Anolis OS 23	2 3.10.1	身份、Python、compileall 通过

历史证据为 evaluation/results/os_matrix.json。9 月 22 日最新验收记录在 artifacts/video_recording_v3/runs/tests_provenance.json、openkylin_provenance.json、anolis_provenance.json；openEuler 的 pytest 用时 4.99 秒、退出码 0。openKylin 和 Anolis 此次没有重新记录完整 pytest，不能将 44 项测试通过扩展为三个系统的共同新结论。202 条警告按最新日志如实保留，不与历史 17 条警告混写。

同批 openEuler 单场景演示读取 scenario_10_0.json 的 10 个任务、16 条依赖：HEFT makespan 为 0.455228436，Adaptive 为 0.390745507，ratio=0.858350393，缩短 14.1650%。输出 validation.valid=true，模型载入 1.144 秒，调度 1.310 秒，峰值进程 RSS 为 338.5 MiB。其结果及命令来源为 artifacts/video_recording_v3/runs/adaptive.json 和 adaptive_provenance.json，HEFT 对照见同目录 heft.json。该例说明真实接口可运行，14.1650% 是单例收益，不能替代正式主实验的 8.3559%。

4.2.7 结论适用范围

现有证据支持：在固定云边端资源模型下，最终系统在常规验证、宽并行专项和 30~60 任务规模测试中保持对 HEFT 的非回归，并以额外计算时间换取 makespan 改善。HEFT 锚点与 best-only 选择提供同一物理模型下的非回归机制；这一机制保证候选选优的下限，不保证对真实平台测量误差或动态扰动仍有相同比例收益。

当前实验是已知 DAG 的离线仿真，未覆盖动态任务到达、资源离线、失败重试、容器冷启动、多租户公平性与能耗。主统计仅验证固定模型的推理随机性，未验证多训练种子的稳健性；结构专项样本较少，模型超过容量时依赖启发式降级。后续应扩大独立测试集、记录真实集群通信与运行开销，并在原生系统和端设备上继续验证。报告将这些事项列为待验证边界，不写为已完成成果。

第 5 章 实现难点说明

5.1 调度物理一致性与合法性保障

难点在于任务顺序、资源竞争、通信等待和空隙插入共同决定结果。只检查拓扑顺序不能保证时间线正确，只对非法动作加惩罚也不能保证输出可执行。解决办法是统一资源代价与插入函数，在构造阶段屏蔽非就绪任务，在搜索阶段检查全部依赖并使用 Kahn 修复，在输出阶段再次验证任务是否齐全、资源是否合法、开始时间是否非负、结束时间是否早于开始时间、同资源任务是否重叠、每条依赖是否满足通信就绪时间。

对应回归测试将同一合法顺序在环境、HEFT 和重放器中执行，检查资源分配和 makespan 一致；最终命令行输出携带 `validation.valid` 和具体错误列表。该措施同时提高结果可信度和错误可定位性：高性能数字只有在完整合法调度通过验收后才具有意义。

5.2 策略塌缩与异构动作学习

初版 Joint 策略同时决定任务与资源，绝对时间奖励跨实例量纲不同，特征尺度也不一致。诊断中 10 个 DAG 的 180 个任务全部分配到 `cloud_0`，说明模型偏向最快单节点，未形成有效并行。仅改相对奖励后 `mean_ratio` 仍为 1.367762，不能把奖励修改单独认定为成功。

项目先通过动作分布、批量 mask 和小数据过拟合定位问题，再依次归一化连续观测、改为 task-only 动作、注入 HEFT 残差先验。归一化后的历史比值为 1.293190，Ranked 为 1.081277，Residual 确定性策略为 1.031197。这些阶段说明改进来自观测与学习问题的共同重构；单次策略仍可能落后，因此最终使用多样采样和约束精修。历史结果文件均保留在 `evaluation/results/`。

5.3 搜索质量、拓扑约束与计算代价平衡

单任务移动会影响后续 EFT 选择与整条资源时间线，局部看似合理的排序也可能违反长距离依赖。项目对每个候选检查全部边，再完整重放，使改善判据直接对应最终 makespan。LNS 则释放少量位置锚点，用零入度修复跨越单任务邻域，随后进行局部精修。best-only 接受与 incumbent 保留保障已获得方案不被更差搜索结果覆盖。

这一设计需要重复评估候选，不能只报告完成时间改善而忽略调度器本身耗时。项目用三档工作量、软截止、候选阶段计时与邻居评估计数控制投入，并保留计算量对齐的 Best-of-128 对照。方法价值是有限推理预算内改善离线 workflow 方案，当前实现不具备硬实时期限保证。

5.4 宽并行结构与固定模型容量

训练分布中的有效排序经验不一定适用于大量同层就绪任务，固定填充网络也不能直接接收任意规模图。项目先以结构化场景验证宽并行短板，再依据层宽比例引入宽度和成本候选；所有分支共享物理规则，并与 HEFT 锚点统一选优，避免新增候选造成性能回归。

超过 30 个任务时显式跳过模型，继续启发式和 LNS；输出注明跳过原因和胜出来源。30~60 任务实验由此验证的是完整系统的容量安全与搜索扩展能力。它不能证明该 checkpoint 在 40~60 任务上实现神经策略泛化，这一区别在理论、结果和使用说明中保持一致。

5.5 随机实验复现与跨平台证据边界

采样与搜索包含多种随机源，单次优秀结果容易被误当作稳定提升。项目固定验证场景、资源配置与 checkpoint，保存 Python、NumPy、PyTorch 和搜索种子，使用五轮配对推理重复。最终主实验的 0.916441 ± 0.000984 中，标准差描述五轮场景均值的波动；它不是 100 个独立训练结果，也不是全部单场景比值的标准差。配对 t 检验的样本单位为五轮均值，结论限于该模型、固定场景集和评测协议。

跨平台复现通过锁定通用依赖、单独安装 CPU PyTorch、容器构建和机器可读日志完成。openEuler、openKylin 与 Anolis OS 矩阵验证了发行版用户态的构建、测试和场景生成；Docker Desktop 的 WSL2 共享宿主内核，证据不能外推为三种发行版原生内核或物理设备调度验证。输入与模型哈希、环境版本、测试输出和运行状态一起保存，便于解释版本变化及性能差异。

第 6 章 总结

6.1 完成情况与主要收获

项目完成了可配置 DAG 环境、统一策略接口、HEFT/MILP/Hybrid 基线、MaskablePPO 训练、Residual 多样化推理与合法搜索、Adaptive 选优、固定评测、配对统计、CLI 和跨平台复现入口。最终方案在原始 20 场景上五次重复达到 $\text{mean_ratio}=0.916441 \pm 0.000984$ ，每轮 20/20 反超 HEFT。结论来自现存原始结果，报告同时保留纯采样对照、历史失败路线和分布外结果，具体协议与统计见实验测试章。

主要收获是明确了学习模型与约束求解器的职责：模型提供排序偏好和候选多样性，统一 EFT 规则建立可比时间线，搜索在合法邻域中改善，HEFT 锚点与输出校验形成安全边界。观测归一化、动作空间设计和相对奖励的诊断说明，训练曲线并不能单独证明调度有效，必须用独立场景、强基线和原始计划逐项核查。

6.2 适用边界与后续工作

当前问题是任务依赖与代价预先已知的静态、非抢占 DAG 调度，不覆盖动态到达、资源故障、迁移、冷启动、多租户公平性和能耗。正式 Residual 模型输入容量为 30 任务；30~60 任务扩展依靠容量检查、HEFT 锚点和搜索，不能等价为网络自身的规模泛化。多次采样与 LNS 使用秒级计算换取质量，适合离线批调度，尚不适合硬实时毫秒级控制。

后续优先开展多训练种子和封存结构测试集验证，训练对宽并行与异构变化更稳定的 checkpoint；研究集合或图编码与增量 makespan 计算，降低重复重放成本；在国产操作系统虚拟机或物理机保存原生内核运行证据；再通过执行器、监控和资源遥测接入真实 workflow。动态负载与故障扩展应建立新的实验协议，继续保留当前静态结果作为可复核基准。

6.3 开源组件与知识来源

项目感谢 HEFT、PPO 和动作屏蔽等研究提供的方法基础，以及 PyTorch、Gymnasium、Stable-Baselines3、SB3-Contrib、NetworkX、NumPy、SciPy、PuLP、Matplotlib 等开源项目提供的能力。第三方依赖与自主实现的具体边界在代码原创说明中列出，方法来源采用著者一出版年引用；引用算法思想不等同于复制第三方源码，后续修改仍需遵守相应许可证和来源说明。

参 考

Topcuoglu H, Hariri S, Wu M Y (2002) . Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing[J]. IEEE Transactions on Parallel and Distributed Systems, 2002, 13(3): 260 – 274. DOI: 10.1109/71.993206.
<https://doi.org/10.1109/71.993206>.

Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O (2017) . Proximal Policy Optimization Algorithms[EB/OL]. arXiv:1707.06347, 2017[2026-10-02].
<https://arxiv.org/abs/1707.06347>.

Huang S, Ontañón S (2020) . A Closer Look at Invalid Action Masking in Policy Gradient Algorithms[EB/OL]. arXiv:2006.14171, 2020[2026-10-02]. <https://arxiv.org/abs/2006.14171>.

Raffin A, Hill A, Gleave A, Kanervisto A, Ernestus M, Dormann N (2021) . Stable-Baselines3: Reliable Reinforcement Learning Implementations[J]. Journal of Machine Learning Research, 2021, 22(268): 1 – 8. <https://www.jmlr.org/papers/v22/20-1364.html>.

Ropke S, Pisinger D (2006) . An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows[J]. Transportation Science, 2006, 40(4): 455 – 472. DOI: 10.1287/trsc.1050.0135.
<https://pubsonline.informs.org/doi/10.1287/trsc.1050.0135>.

NetworkX Developers (无日期) . NetworkX Documentation[EB/OL]. [2026-10-02].
<https://networkx.org/documentation/stable/>.

Farama Foundation (无日期) . Gymnasium Documentation[EB/OL]. [2026-10-02].
<https://gymnasium.farama.org/>.

NumPy Developers (无日期) . NumPy Documentation[EB/OL]. [2026-10-02].
<https://numpy.org/doc/>.

PyYAML Developers (无日期) . PyYAML Documentation[EB/OL]. [2026-10-02].
<https://pyyaml.org/wiki/PyYAMLDocumentation>.

PyTorch Contributors (无日期) . PyTorch Documentation[EB/OL]. [2026-10-02].
<https://docs.pytorch.org/docs/stable/>.

Stable-Baselines3 Contrib Developers（无日期）. Maskable PPO Documentation[EB/OL]. [2026-10-02]. https://sb3-contrib.readthedocs.io/en/master/modules/ppo_mask.html.

PuLP Developers（无日期）. Optimization with PuLP[EB/OL]. [2026-10-02]. <https://coin-or.github.io/pulp/>.

COIN-OR（无日期）. Cbc: COIN-OR Branch-and-Cut Solver[CP/OL]. [2026-10-02]. <https://github.com/coin-or/Cbc>.

SciPy Developers（无日期）. SciPy[EB/OL]. [2026-10-02]. <https://scipy.org/>.

Matplotlib Developers（无日期）. Matplotlib Documentation[EB/OL]. [2026-10-02]. <https://matplotlib.org/stable/>.

pytest Developers（无日期）. pytest Documentation[EB/OL]. [2026-10-02]. <https://docs.pytest.org/en/stable/>.

TensorBoard Developers（无日期）. TensorBoard[CP/OL]. [2026-10-02]. <https://github.com/tensorflow/tensorboard>.

tqdm Developers（无日期）. tqdm Documentation[EB/OL]. [2026-10-02]. <https://tqdm.github.io/>.

Rich Developers（无日期）. Rich Documentation[EB/OL]. [2026-10-02]. <https://rich.readthedocs.io/en/stable/>.